วงแหวนจอมมาร (demon_ring)



คาถาที่ซ่อนอยู่ในหอคัมภีร์เผยว่า ทางไปสู่จอมมารวังวนถูกปิดด้วยวงแหวนตราหยกหมายเลข 1 ถึง N ที่ร้อยเป็นวงกลม ต้องทำพิธีปลดตราทีละดวง: เริ่มนับจากตราที่กำหนดเป็นดวงที่ 1 ตามทิศ `next` นับถึงดวงที่ k แล้วปลดดวงนั้น จากนั้นเริ่มนับ 1 ใหม่ที่ดวงถัดไป ทำซ้ำจนเหลือเพียงดวงเดียวคือ "ตราผู้รอด" ที่จะเปิดประตู จอมมารใช้พลังขยายเลข k ให้ใหญ่ถึงพันล้าน อรุณและหลินจึงต้องรู้ว่าการนับวนรอบที่เกินจำนวนตราที่เหลือ ไม่จำเป็นต้องเดินให้ครบทุกก้าว

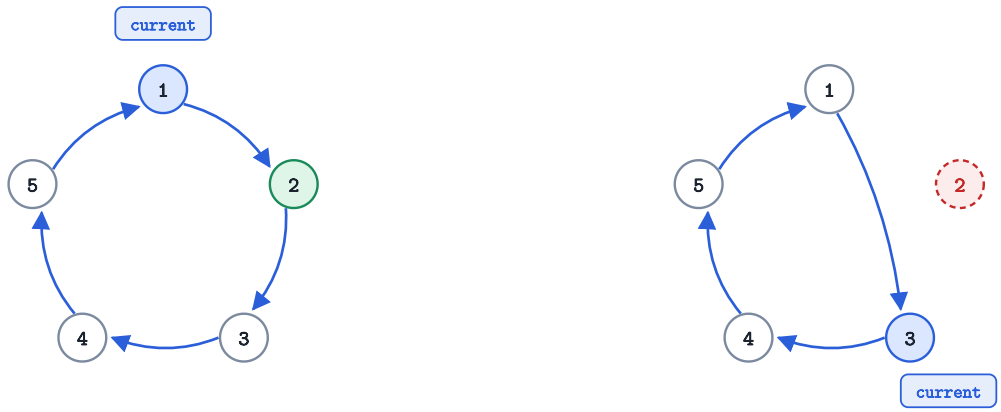
สิ่งที่ฝึกในด่านนี้: ลิสต์วงกลม (circular list) และการนับวน (Josephus): ปิดวง, ลบกลางวง, ลดก้าวด้วยเศษ

รายละเอียดการเขียนโปรแกรม

เขียนฟังก์ชันที่ประกาศในไฟล์ `demon_ring.h` ต่อไปนี้ โดยไม่ต้องเขียน `main` และไม่ต้องอ่านหรือพิมพ์ข้อมูลทางอินพุตและเอาต์พุตมาตรฐาน (โปรแกรมตรวจของผู้จัดทำหน้าที่นั้น)

```
struct Node { int data; Node* next; };
Node* buildRing(int n);
std::vector<int> breakRing(Node*& current, int k);
```

- `buildRing(n)`: สร้างวงกลมตราหมายเลข 1, 2, ..., n เชื่อมกันด้วย `next` โดย `next` ของโหนด n ชี้กลับโหนด 1 แล้วคืนพอยน์เตอร์ของโหนดหมายเลข 1 (รับประกัน $n \geq 1$ ถ้า $n = 1$ โหนดนั้นชี้ `next` ไปที่ตัวเอง)
- `breakRing(current, k)`: `current` ชี้โหนดเริ่มนับ (นับเป็นดวงที่ 1) เดินตาม `next` จนถึงดวงที่ k ลบและ `delete` ดวงนั้น แล้วเริ่มนับ 1 ใหม่ที่ดวงถัดไป ทำซ้ำจนเหลือโหนดเดียว คืนเวกเตอร์ของ `data` ที่ถูกลบตามลำดับ ($N - 1$ ค่า) เมื่อจบ `current` ต้องชี้โหนดที่รอด
- `buildRing` ต้องสร้างโหนดด้วย `new` ครบหมายเลข 1..N เรียงตาม `next` ให้เป็นวงกลมสมบูรณ์ และคืนโหนดหมายเลข 1
- `breakRing` ต้อง `delete` ทุกโหนดที่ลบหนึ่งครั้งพอดี ไม่ `delete` โหนดที่รอด ห้ามเปลี่ยน `data` และเมื่อจบโหนดที่รอดต้องชี้ `next` มาที่ตัวเอง (`current->next == current`)
- k ถึง 10^9 แต่จำนวนตราที่เหลือมีน้อยกว่ามาก การเดินทีละก้าวจริงทั้ง $k - 1$ ก้าวในทุกรอบจะช้าเกินเวลา



รูปที่ 1 วงแหวน $N = 5$ และ $k = 2$: เริ่มนับที่ตรา 1 (นับเป็นดวงที่ 1) ดวงที่ 2 คือตรา 2 (สีเขียว) จึงถูกปลด แล้ว `current` ย้ายไปตรา 3 เพื่อเริ่มนับ 1 ใหม่

การส่งงาน: ดาวน์โหลดไฟล์แนบ `demon_ring.cpp` (ไฟล์โครง) และ `demon_ring.h` จากหน้าโจทย์ แล้วเขียนฟังก์ชันลงในไฟล์ `demon_ring.cpp` ให้เสร็จ จากนั้นส่งเฉพาะไฟล์ `demon_ring.cpp` ห้ามแก้ไขไฟล์ `demon_ring.h` ระบบจะคอมไพล์ไฟล์ของคุณพร้อมกับ `grader.cpp` ของผู้จัด ไฟล์ `attachment.zip` มีโปรแกรมตรวจสำหรับทดลองในเครื่อง (`g++ -std=c++17 -O2 grader.cpp demon_ring.cpp -o demon_ring`) ซึ่งตรวจการใช้ `new` / `delete` และความถูกต้องของโหนดให้เหมือนกับระบบจริง

ตัวอย่างการเรียกฟังก์ชัน

การเรียกฟังก์ชัน	ผลที่ได้
`buildRing(5)`	ได้วง $1 \to 2 \to 3 \to 4 \to 5 \to$ (กลับ 1) และคืนโหนดหมายเลข 1
`breakRing(current, 2)` เมื่อ $N = 5$	คืน `{2, 4, 1, 5}` และ `current` ชี้โหนด 3
`breakRing(current, 3)` เมื่อ $N = 7$	คืน `{3, 6, 2, 7, 5, 1}` และ `current` ชี้โหนด 4
`breakRing(current, 8)` เมื่อ $N = 3$	คืน `{2, 1}` และ `current` ชี้โหนด 3 (นับวนเกินรอบได้)
`breakRing(current, 1000000000)` เมื่อ $N = 1$	คืน `{}` และ `current` ชี้โหนด 1 โดยไม่ต้องนับอะไรเลย

ตารางแสดงตัวอย่างการเรียกที่แยกจากกัน แต่ละแถวเริ่มจากวงที่สร้างใหม่ด้วย `buildRing(N)` และ `current` ชี้โหนดหมายเลข 1

ข้อจำกัด

- $1 \le Q \le 20$
- $1 \le N \le 5\ 000$ และ $1 \le k \le 10^9$ ในแต่ละชุด
- ผลรวมของ N ทุกชุดในกรณีทดสอบเดียวกันไม่เกิน 30 000
- เวลา 2 วินาที และหน่วยความจำ 256 MiB ต่อกรณีทดสอบ

ปัญหาย่อย

1. (20 คะแนน) $N \leq 10$ และ $k \leq 10$ ทุกชุด
2. (30 คะแนน) $N \leq 1\ 000$ และ $k \leq 1\ 000$ ทุกชุด
3. (50 คะแนน) ไม่มีข้อจำกัดเพิ่มเติม (k ถึง 10^9)

แต่ละปัญหาย่อยคิดคะแนนแยกกัน จะได้รับคะแนนของปัญหาย่อยเมื่อผ่านกรณีทดสอบทุกกรณีในปัญหาย่อยนั้น

รูปแบบข้อมูลสำหรับโปรแกรมตรวจตัวอย่าง

บรรทัดแรก: จำนวนเต็ม Q คือจำนวนชุดข้อมูล
จากนั้นอีก Q บรรทัด แต่ละบรรทัดมีจำนวนเต็ม N และ k
โปรแกรมตรวจเรียก `buildRing(N)` แล้วเรียก `breakRing(current, k)` โดยให้ `current` ชี้โหนดหมายเลข 1 ต่อหนึ่งชุดพิมพ์ 2 บรรทัด: บรรทัดแรกคือ $N - 1$ ตามด้วยหมายเลขของตราที่ถูกปลดตามลำดับ บรรทัดที่สองคือหมายเลขของตราผู้รอด (ค่า `data` ของโหนดที่ `current` ชี้ตอนจบ)

ตัวอย่างข้อมูลนำเข้าและผลลัพธ์

ข้อมูลนำเข้า 1	ผลลัพธ์ 1
2 5 2 7 3	4 2 4 1 5 3 6 3 6 2 7 5 1 4

$N = 5, k = 2$ ปลดตรา 2, 4, 1, 5 ตามลำดับ เหลือตรา 3 · $N = 7, k = 3$ ปลดตรา 3, 6, 2, 7, 5, 1 เหลือตรา 4
ตัวอย่างนี้สอดคล้องกับปัญหาย่อยที่ 1, 2, 3

ข้อมูลนำเข้า 2	ผลลัพธ์ 2
3 1 5 3 1 2 3	0 1 2 1 2 3 1 1 2

$N = 1$ ไม่มีตราให้ปลด · $k = 1$ ปลดตราที่ 1, 2 ตามลำดับ (นับ 1 ก้าวคือดวงแรกเสมอ) เหลือตรา 3 · $N = 2, k = 3$ นับวนรอบ: 1, 2, 1 จึงปลดตรา 1 ก่อน เหลือตรา 2
ตัวอย่างนี้สอดคล้องกับปัญหาย่อยที่ 1, 2, 3